

SALESTORM

Selling exactly 100 units to 10,000 buyers in one second

Caches can say **NO**. Only the database can say **YES**.

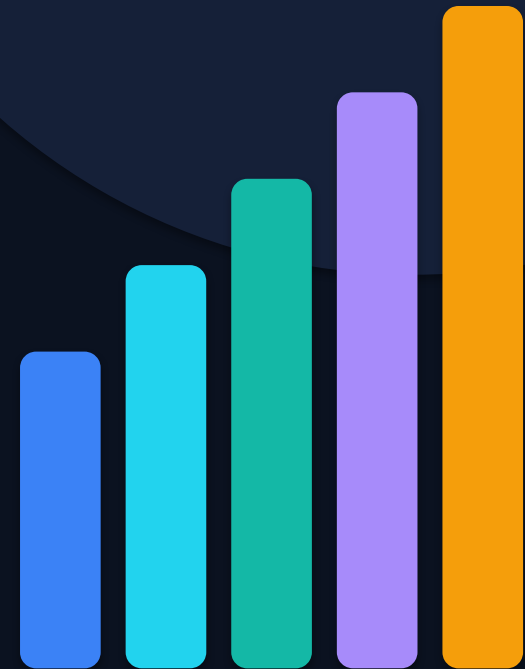
Cache-Hit Ladder

0 oversold

Exactly-once charge

Survives 30 s outage

Flash-sale e-commerce platform • HLD • LLD • DB • API • Reliability • Proof



The problem in numbers

100

units in stock

10,000

buyers click at once

500k

requests / s at peak

95 / 5

% payments succeed / fail

2%

duplicate requests (double clicks, retries)

30 s

Order Service outage mid-sale

Win condition: exactly 100 sold, nobody charged twice, no paid order lost — under all six conditions at once.

What breaks in a naive design

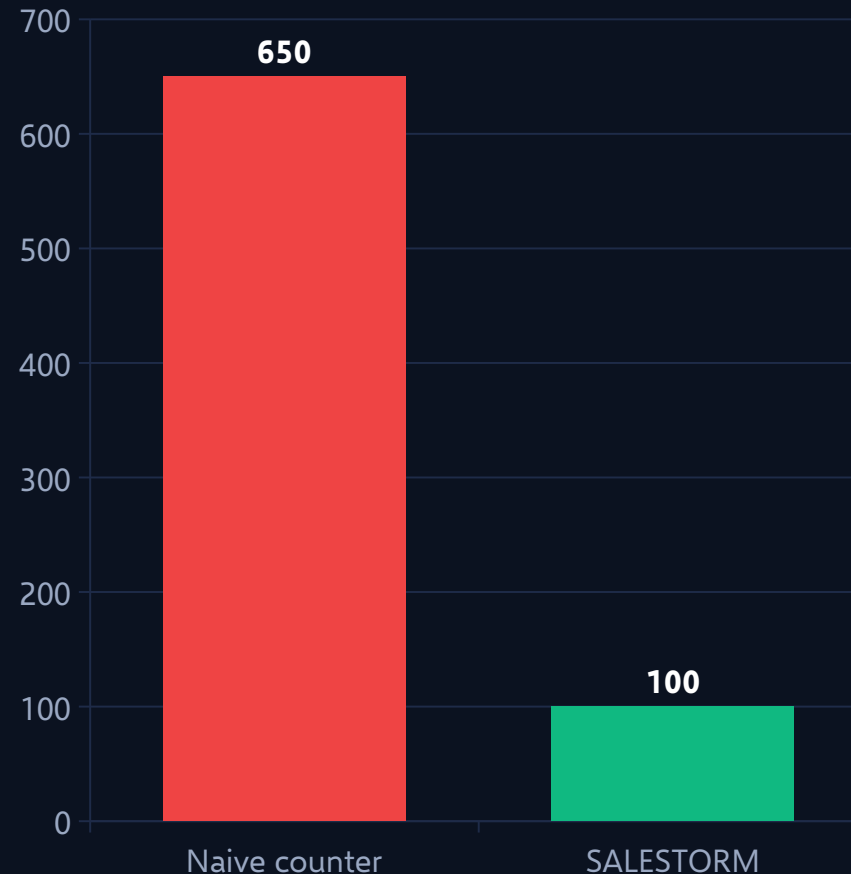
Naive flow

1. SELECT stock
2. if stock > 0: INSERT order
3. UPDATE stock = stock - 1

Why it fails

Read-then-write race (write skew)
Retries double-charge
Order down = paid order lost
One hot row = lock queue

Units sold (stock = 100)



650

sold by the naive design

+550

oversold units = refunds +
angry buyers

Same 10,200 requests, same
simulator

Requirements: hard guarantees vs targets

Hard guarantees — never break



Never sell unit #101

unit rows + CHECK



Never charge twice

Idempotency-Key + UNIQUE



Never lose a paid order

outbox + inbox



One unit per customer

UNIQUE(sale, customer)

Targets — degrade gracefully



p99 reserve < 200 ms

at 10k req/s normal



500k req/s peak

absorbed at CDN + L2



99.95% availability

cell isolation, shedding



Reservation hold 5 min

TTL + sweeper w/ fencing

Unique factor: the Cache-Hit Ladder

L0**Browser**

Countdown, assets, Buy debounce

blocks repeat clicks**L1****CDN + WAF**

Product pages + SOLD_OUT flag

9,921 answered**L2****In-process cache**

Hot product & sold-out flag per pod

163 answered**L3****Redis**

100 admission tokens + idempotency keys

7 rejected • 2 replayed**L4****Postgres**

Unit rows • SKIP LOCKED • UNIQUE

107 reached → 100 sold

Caches can say **NO**. Only the database can say **YES**.

01

High-Level Design

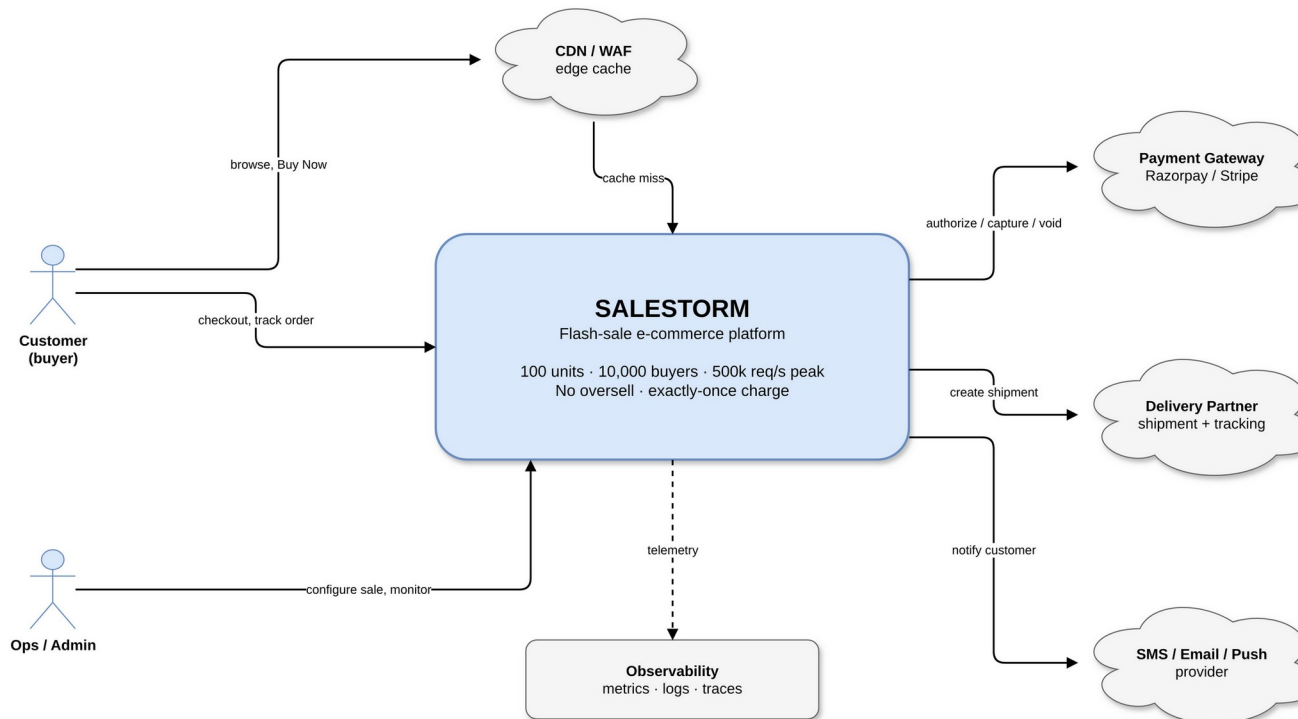
Context → architecture → containers → components → deployment

Owner: Student 1 — System Architect

System context

SALESTORM — System Context (C4 Level 1)

Who uses the system and which external systems it depends on



Actors & externals

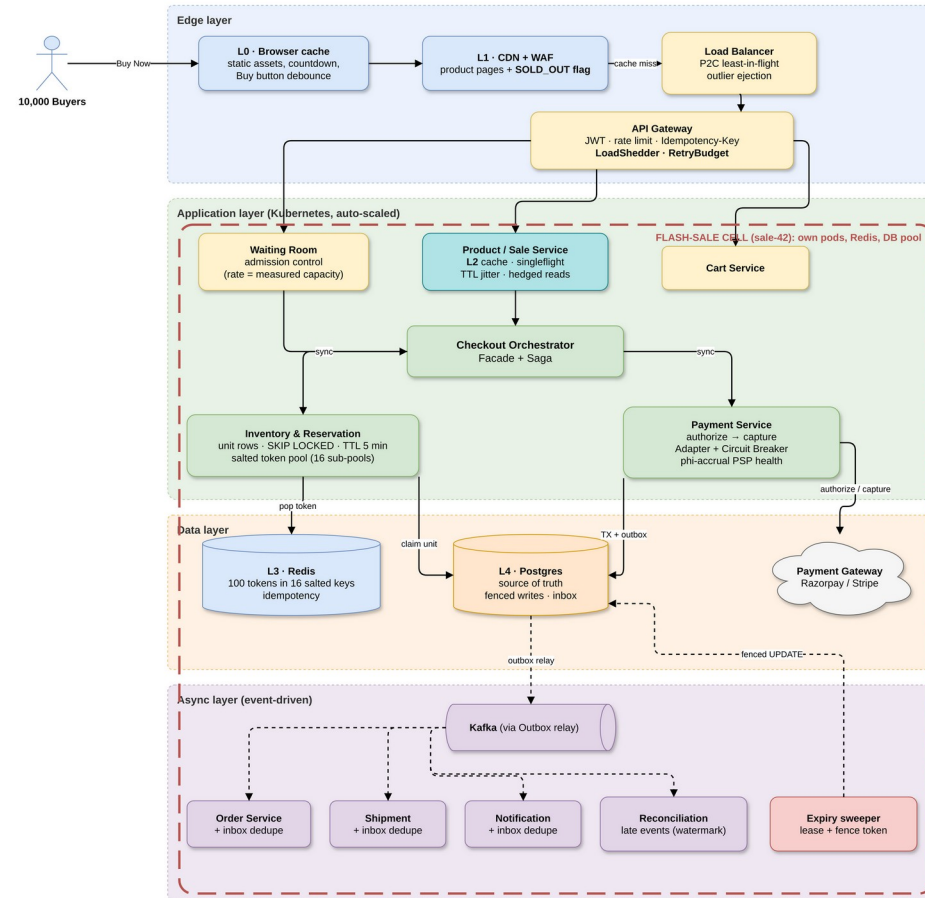
- Buyers (web / mobile)
- Admin: sale config
- Payment gateway (Razorpay / Stripe)
- Delivery partner
- SMS / e-mail provider

One system boundary; every external call is behind an adapter + breaker.

High-level architecture

SALESTORM — High-Level Architecture (Cache-Hit Ladder)

Caches can say NO. Only the database can say YES. - red dashed line = flash-sale cell



Simulation: where 10,200 clicks stopped

9,921
L1 CDN

163
L2 local

7
L3 Redis

2
Replay

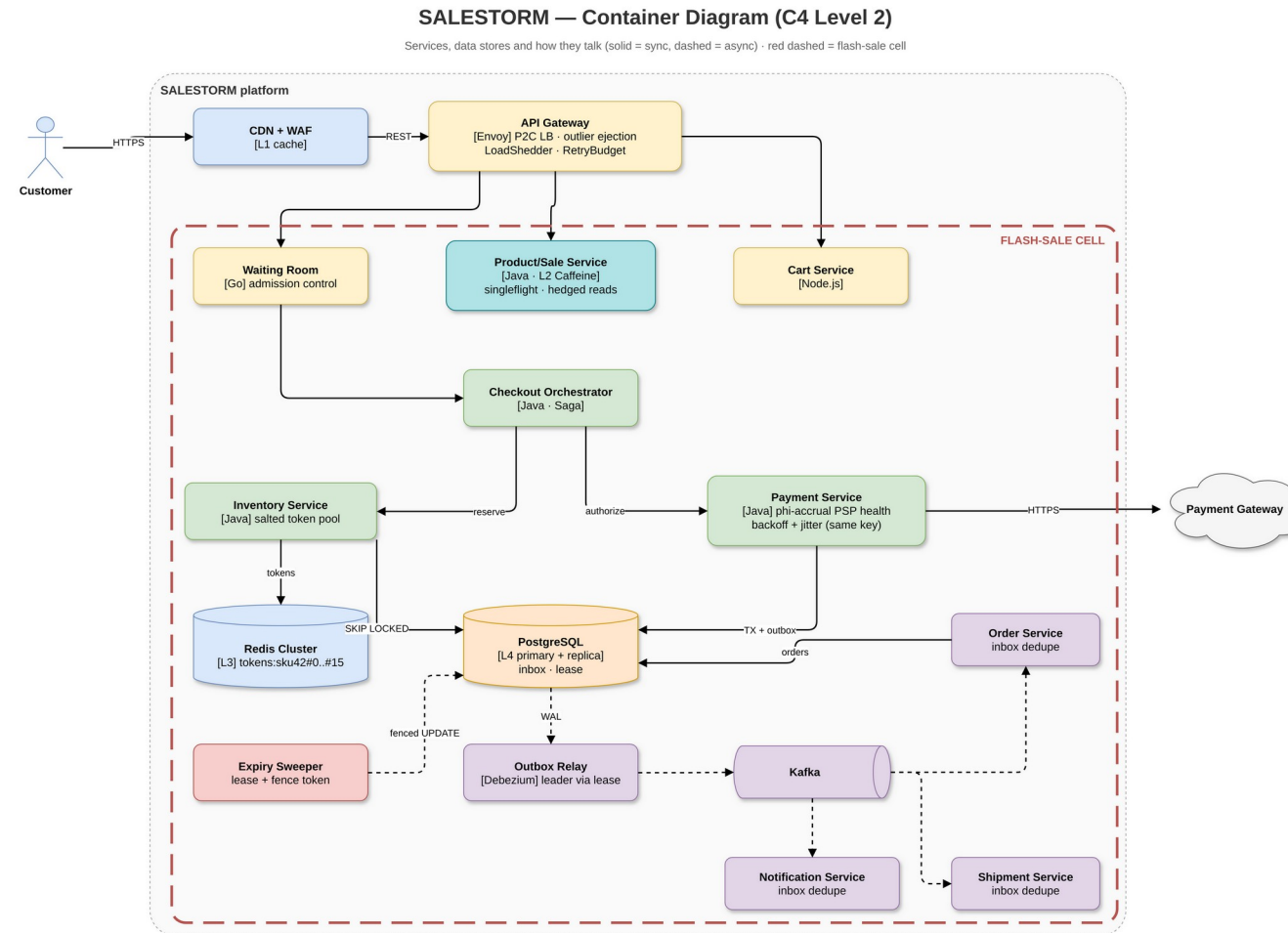
107
Postgres

100
Sold

Caches can say NO.
Only the database can say YES.

Regular catalog / checkout runs in a separate cell, so the sale cannot starve it (and vice versa).

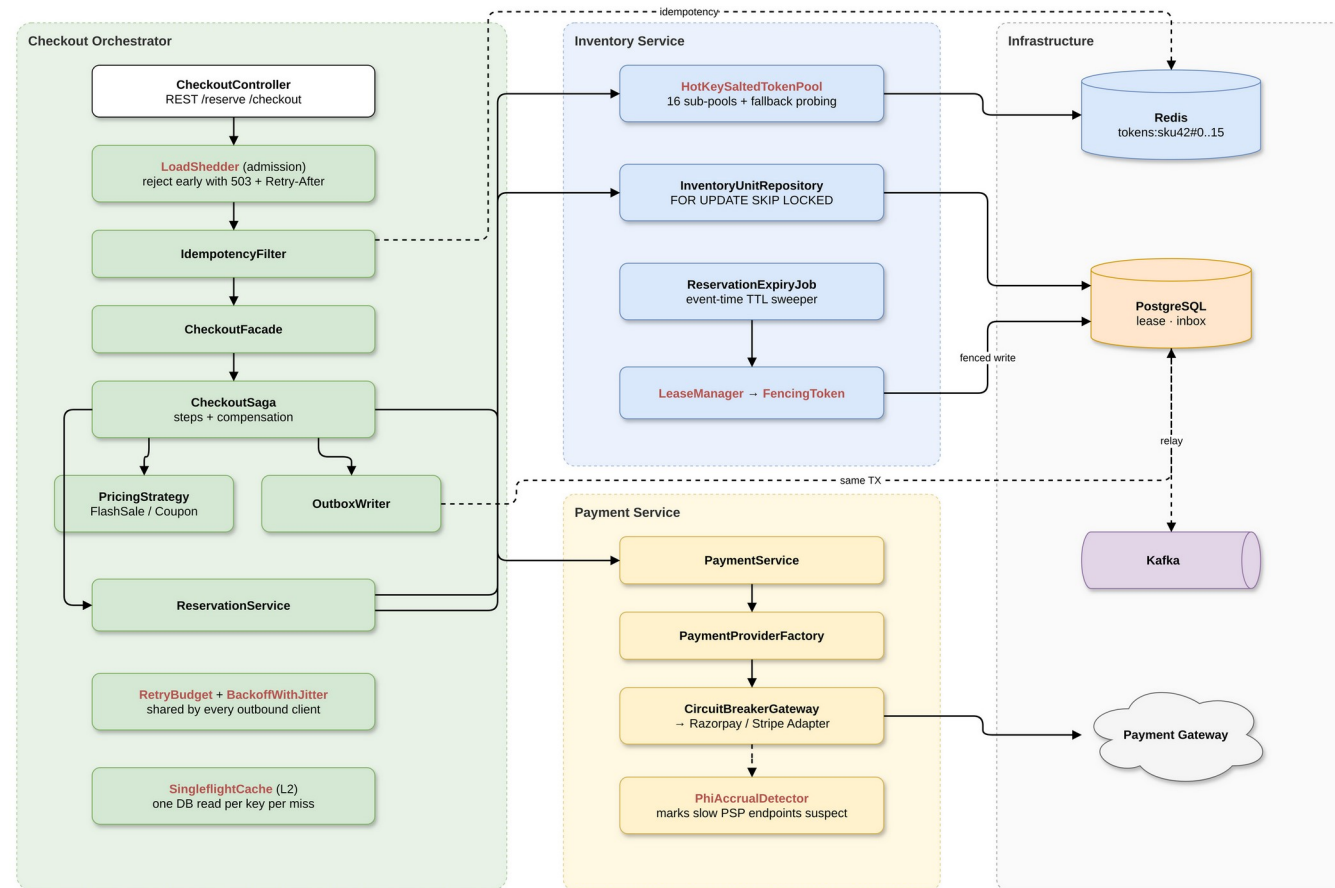
Container view



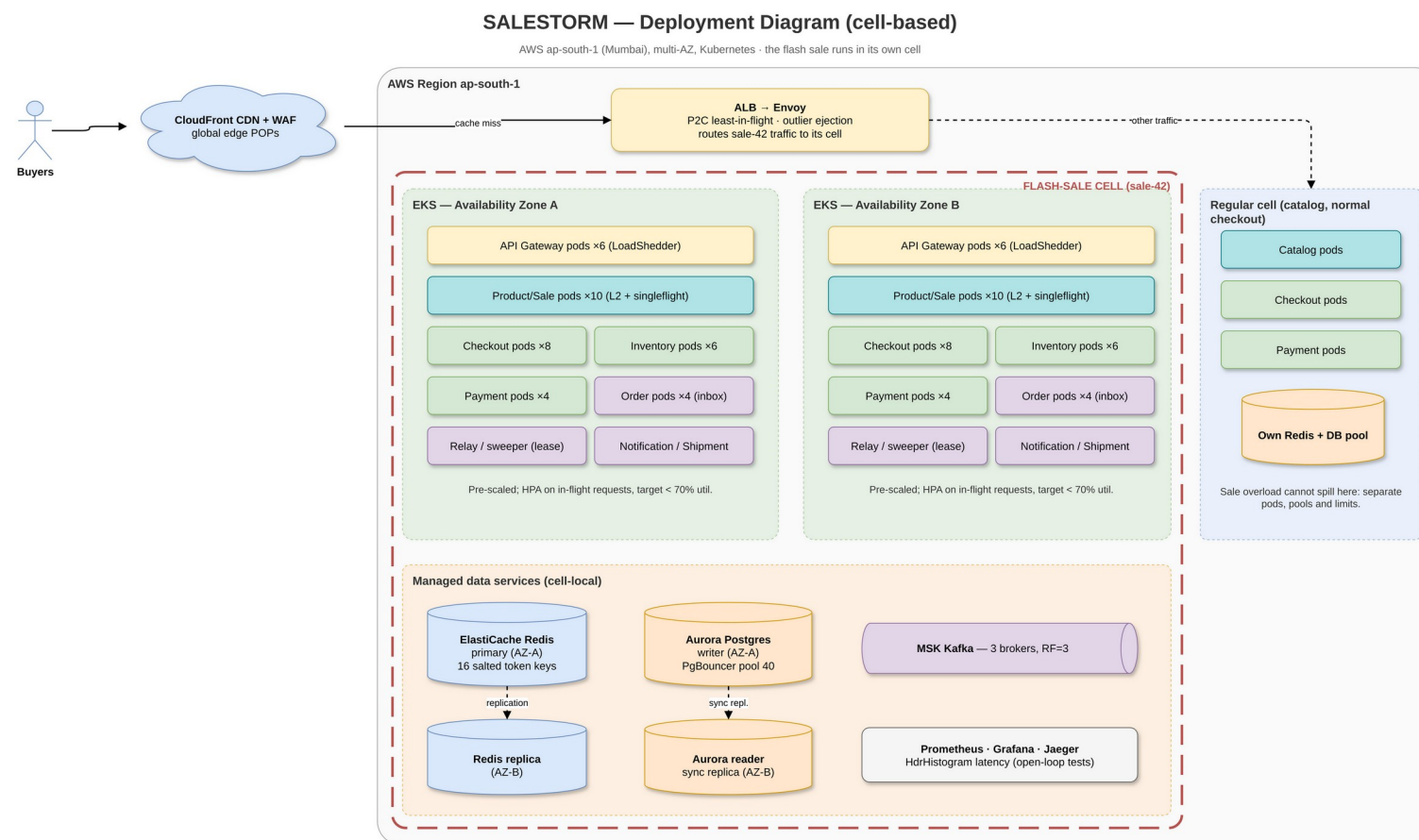
Component view

SALESTORM — Component Diagram (C4 Level 3)

Inside Checkout, Inventory and Payment services (resilience components in bold red text)



Deployment & cell isolation



1

dedicated flash-sale cell

<70%

target utilisation per tier

200

requests in flight (Little's law)

40

Postgres connection pool

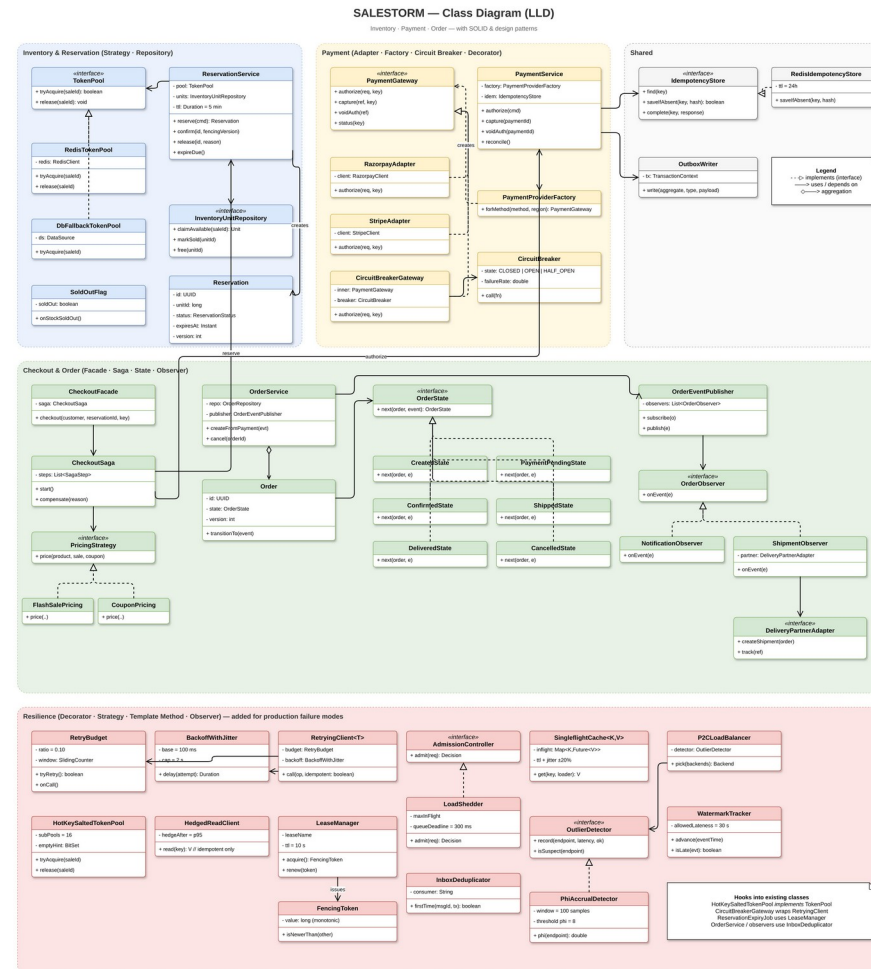
02

Low-Level Design

Classes • SOLID • patterns • sequences • state machines

Owner: Student 2 — LLD & Design Engineer

Class diagram



SOLID mapping

Principle	Where in SALESTORM	Effect
S Single responsibility	ReservationService (stock) • PaymentService (money) • OrderService (lifecycle)	SMS change never touches payment code
O Open / closed	PricingStrategy, PaymentGateway adapters, HotKeySaltedTokenPool	New provider = new class, core unchanged
L Liskov substitution	Razorpay, Stripe, CircuitBreakerGateway honour one contract	PaymentService works with any of them
I Interface segregation	TokenPool, IdempotencyStore, InventoryUnitRepository, OrderObserver	No giant InventoryManager
D Dependency inversion	Services depend on TokenPool / PaymentGateway / LeaseManager	Redis swappable; DB fallback is just another TokenPool

Design patterns and why



Facade

CheckoutFacade

One entry for reserve → pay → order



Saga

CheckoutSaga

Compensate: void auth, release unit



Strategy

PricingStrategy, TokenPool

Swap pricing / admission policy



Factory + Adapter

PaymentProviderFactory

Razorpay ↔ Stripe without code change



Decorator

CircuitBreakerGateway

Breaker wraps any gateway



State

OrderState

Illegal transitions impossible



Observer

OrderEventPublisher

Notify, ship, analytics decoupled



Outbox

outbox_event

Event + data in one transaction

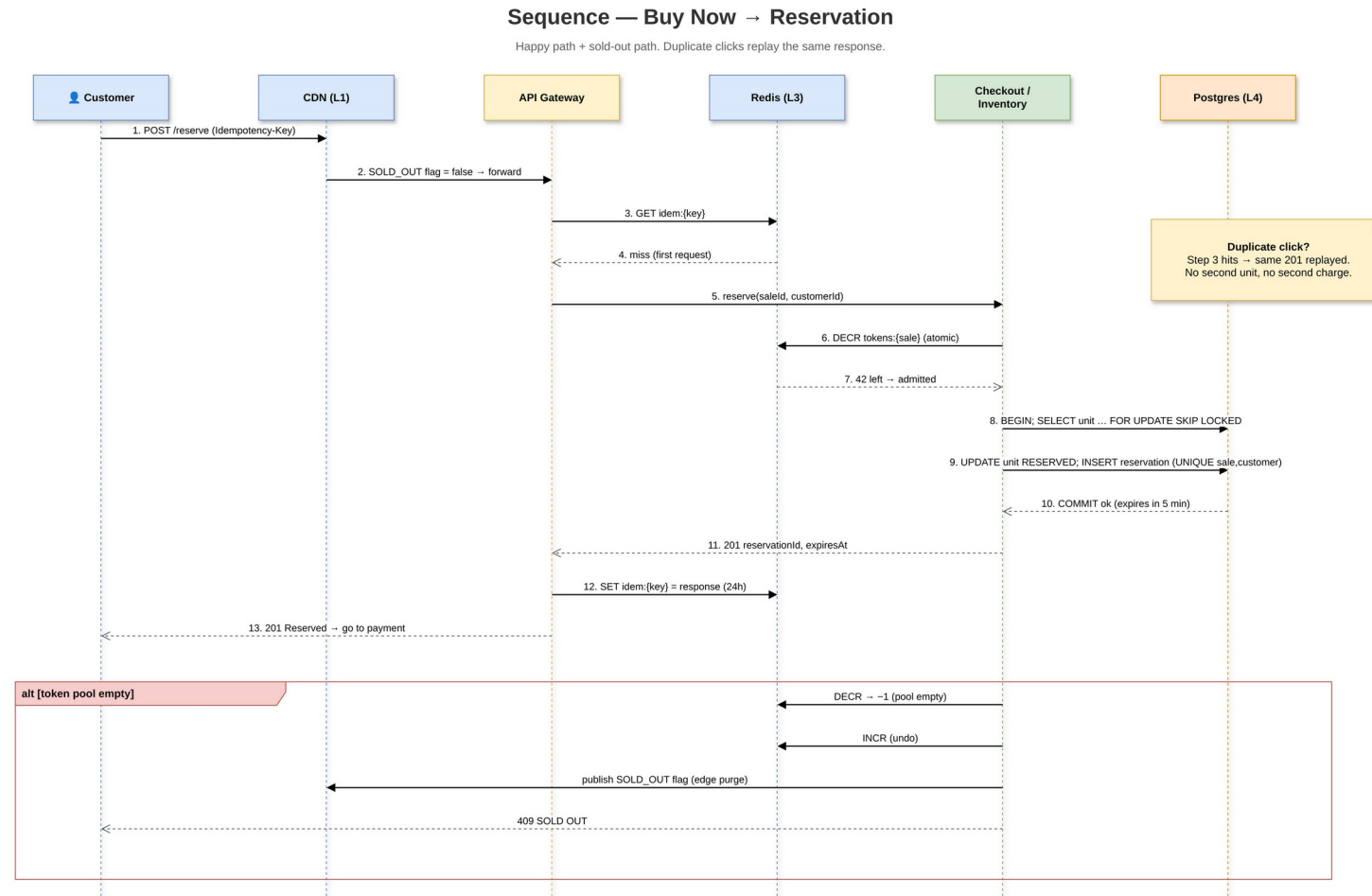


Repository

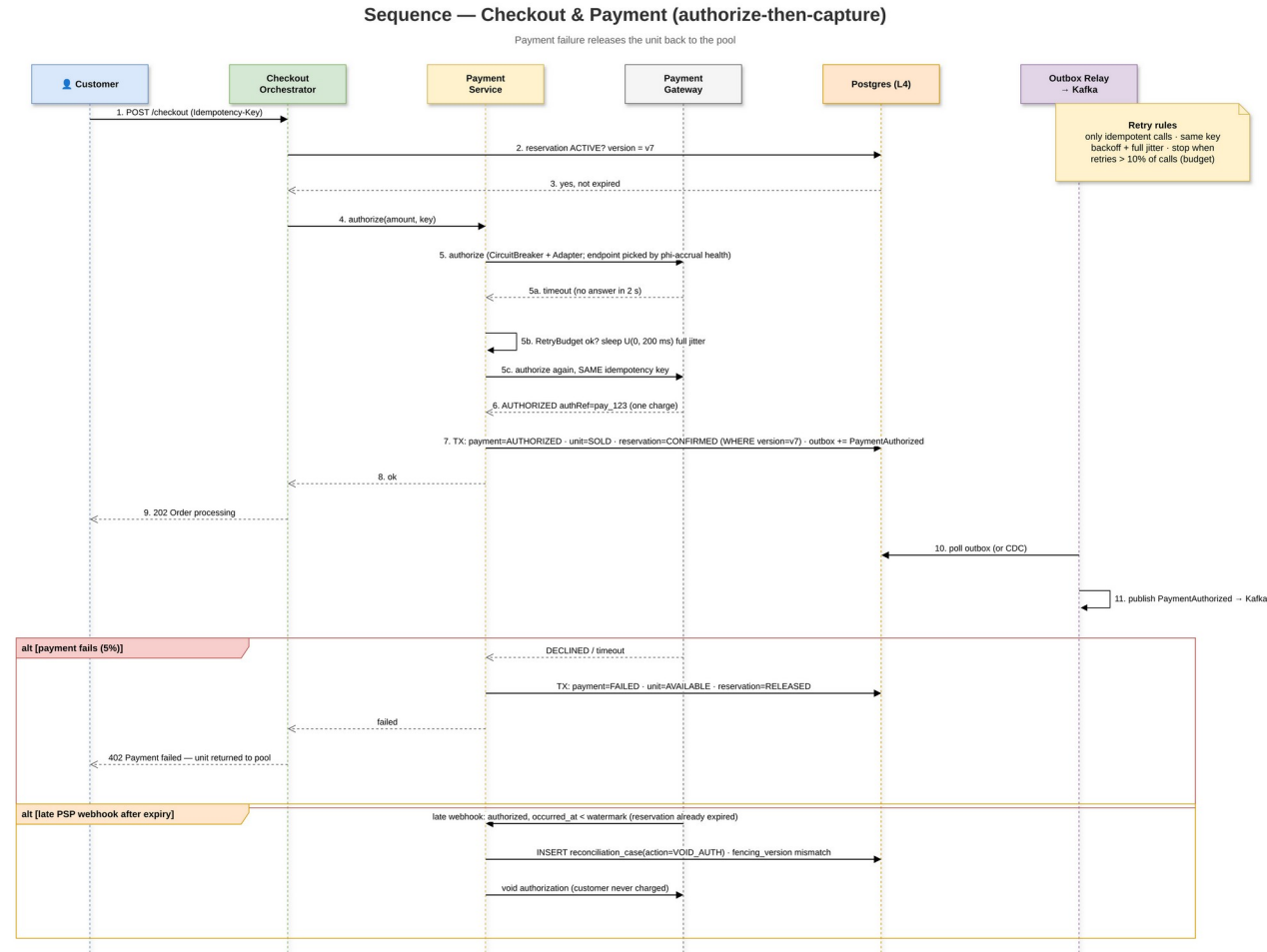
InventoryUnitRepository

SQL hidden behind interface

Sequence: Buy Now → reservation



Payment: authorize, then capture



Idempotency-Key

Duplicate → same stored answer, never a second charge.

Fencing version

Payment after TTL expiry fails the version check → auth voided.

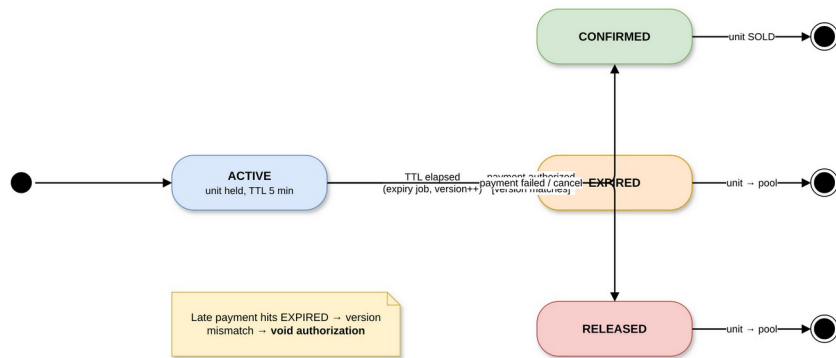
Failure = release

Declined payment returns the unit to the pool for the next buyer.

State machines

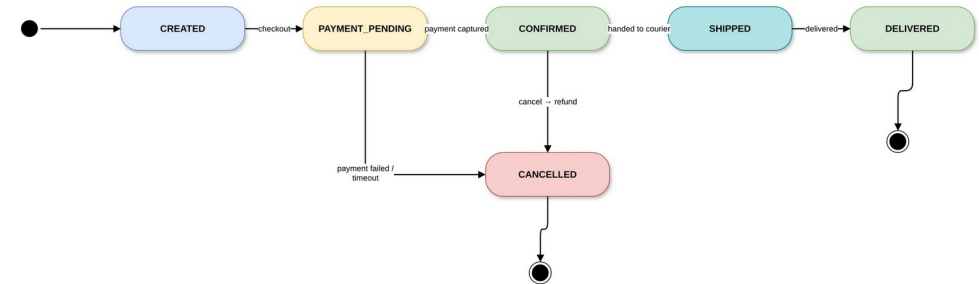
State Machine — Reservation

Fencing version stops an expiry racing a late payment



State Machine — Order (State pattern)

Each state is a class implementing OrderState.next()



03

Data, API & Events

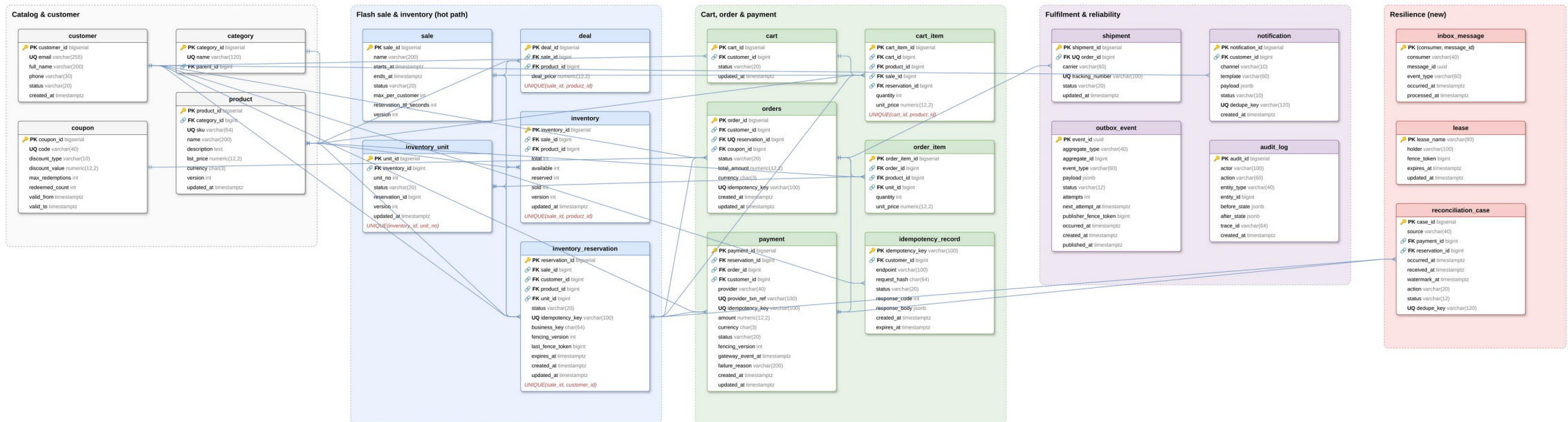
22 tables • idempotent REST API • outbox + inbox

Owner: Student 3 — Data & API Engineer

ER diagram — PostgreSQL, 22 tables

SALESTORM — Entity Relationship Diagram (PostgreSQL, 22 tables)

Generated from G4_DatabaseTools/schema_postgres.sql • 🔑 primary key • 🔗 foreign key



one row per unit

UNIQUE per customer

idempotency_record

outbox (same TX)

inbox + lease

API & idempotency

Method	Endpoint	Key responses
GET	/sales/{id}/stock-status	200 • CDN max-age=1
POST	/sales/{id}/reservations	201 • 200 replay • 409 • 503
POST	/checkout	200 • 409 expired
POST	/payments	202 • 402 • 503 breaker
POST	/payments/webhook	200 • 401 bad HMAC
GET	/orders/{id}	200 order + state
POST	/orders/{id}/cancel	202 • 409 shipped

Idempotency-Key header

Same key → same stored response (200 replay). 2 replays in the sim, 0 double charges.

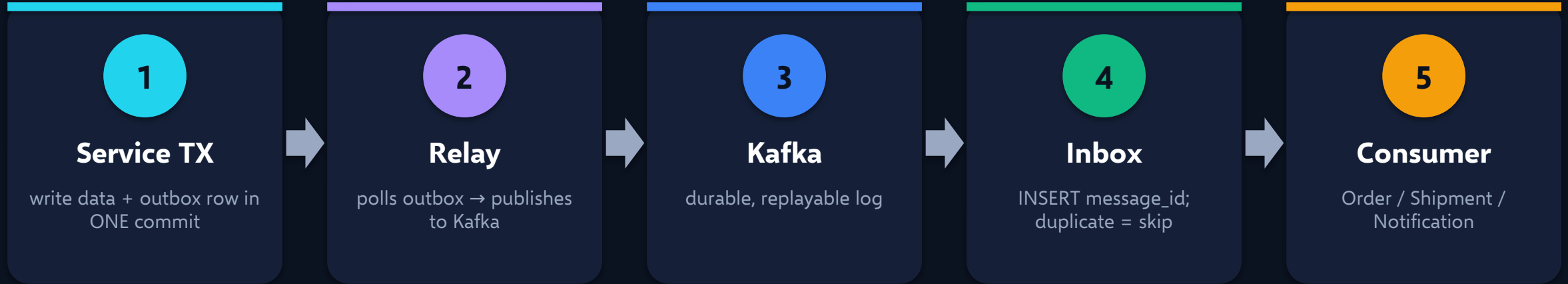
409 SOLD_OUT

Also 409 ALREADY_RESERVED. Fast, cacheable NO — from L1, L2 or L3.

503 + Retry-After

Waiting room under overload: tells clients when to come back instead of hammering us.

Events: transactional outbox + inbox

**0**

events lost — outbox commits with the data

0

duplicate orders with inbox (13 without)

Watermark

event time vs processing time → late events reconciled

04

Concurrency & Reliability

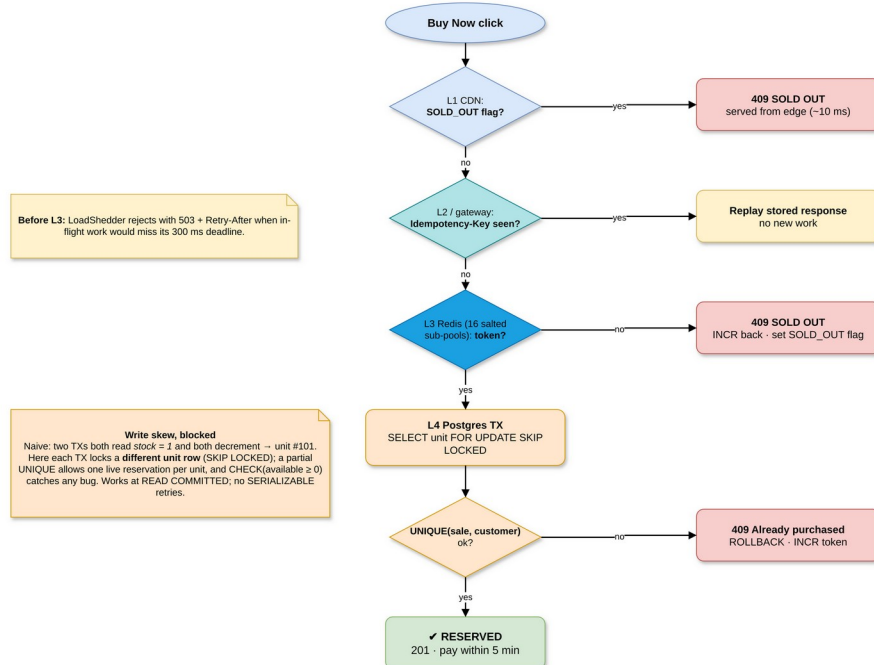
SKIP LOCKED • 14 failure modes • resilience controls • proof

Owner: Student 4 — Reliability Engineer

Concurrency: no oversell, by construction

Concurrency — How one Buy click is decided

Each check can only say NO; the final YES is a Postgres commit



Simulated outcome (10,200 requests)



0 oversold · max 1 charge per customer · 415 invariant checks passed

```
SELECT id FROM inventory_unit
WHERE sale_id=$1 AND status='AVAILABLE'
LIMIT 1 FOR UPDATE SKIP LOCKED;
```

Write skew prevented

Each buyer locks a different row — no read-then-write race.

No hot-row queue

SKIP LOCKED: parallel claims, nobody waits on a lock.

DB is the last line

CHECK(status) + partial UNIQUE(sale, customer).

14 production failure modes we designed for

1	Metastable failure	Load shedder + admission + retry budget	8	Gray failure	Real-traffic health, phi-accrual
2	Retry storm	Idempotent-only retries, jitter, 10% budget	9	Unbalanced backends	Power of two choices (in-flight)
3	Tail latency	Hedged reads at p95 (reads only)	10	Capacity guesswork	Little's law, pool 40, <70% util
4	Cache stampede	Singleflight + TTL jitter	11	Write skew (oversell)	Unit rows + SKIP LOCKED + UNIQUE
5	Hot key (the SKU)	16 salted Redis sub-pools	12	Late events	Event time + watermark → reconcile
6	Duplicate delivery	Outbox + inbox = effectively-once	13	Blast radius	Flash sale in its own cell
7	Split-brain leader	Lease + fencing token checked by Postgres	14	Coordinated omission	Open-loop load tests

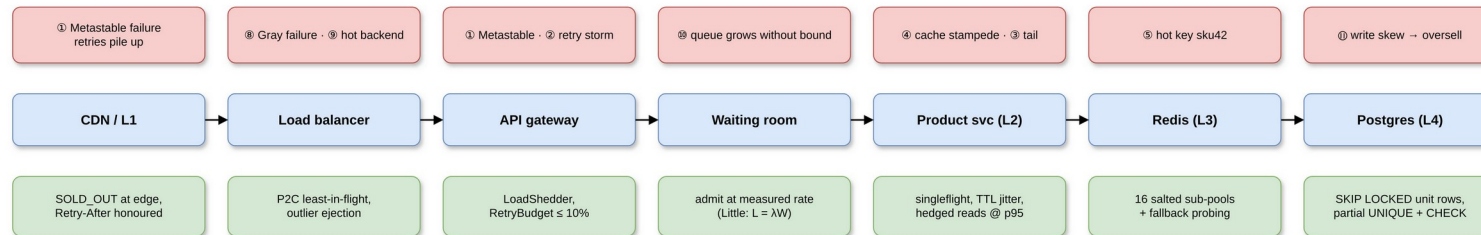
"We would rather reject 5% of requests than let retries turn a blip into a 40-minute outage."

Resilience controls on the request path

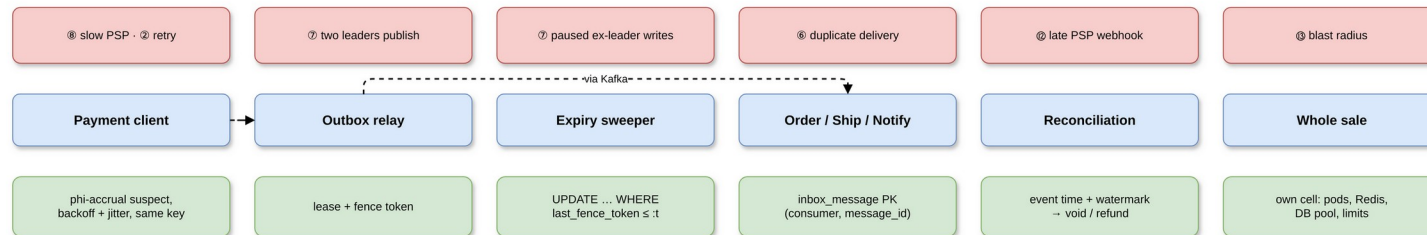
SALESTORM — Failure modes mapped to controls on the request path

Red = what goes wrong · Blue = where it happens · Green = the control we built (numbers match Production_Failure_Modes.md)

Synchronous path (user waiting)



Payment + asynchronous path



⑧ Little's law sizing (numbers from the design)

Reserve path: 10,000 req/s × 20 ms = **200 in flight** → 18 pods × 16 workers = 288 slots (69% busy)
 DB: only ~107 requests ever reach Postgres; 100 tx/s × 30 ms = 3 busy connections → pool of **40** is plenty
 500k req/s peak: 97% stopped at L1 in our simulation → ~15k req/s reach origin; waiting room admits 5k/s

⑧ Load tests that do not lie

Open-loop arrivals (Locust constant arrival rate, JMeter Precise Throughput Timer).
 Latency measured from the **intended** start time → HdrHistogram.
 Our 2 s stall model: closed-loop p99 = **1 ms** (lies), open-loop p99 = **1,902 ms** (truth).

Load shedder → 503 + Retry-After

Retry budget 10% + full jitter

Circuit breaker (Decorator)

Singleflight cache

16 salted token pools

Lease + fencing token

P2C load balancing

Phi-accrual outlier ejection

Inbox dedupe

Proof: simulation of the exact test case

10,200

requests incl. duplicates

107

reached Postgres

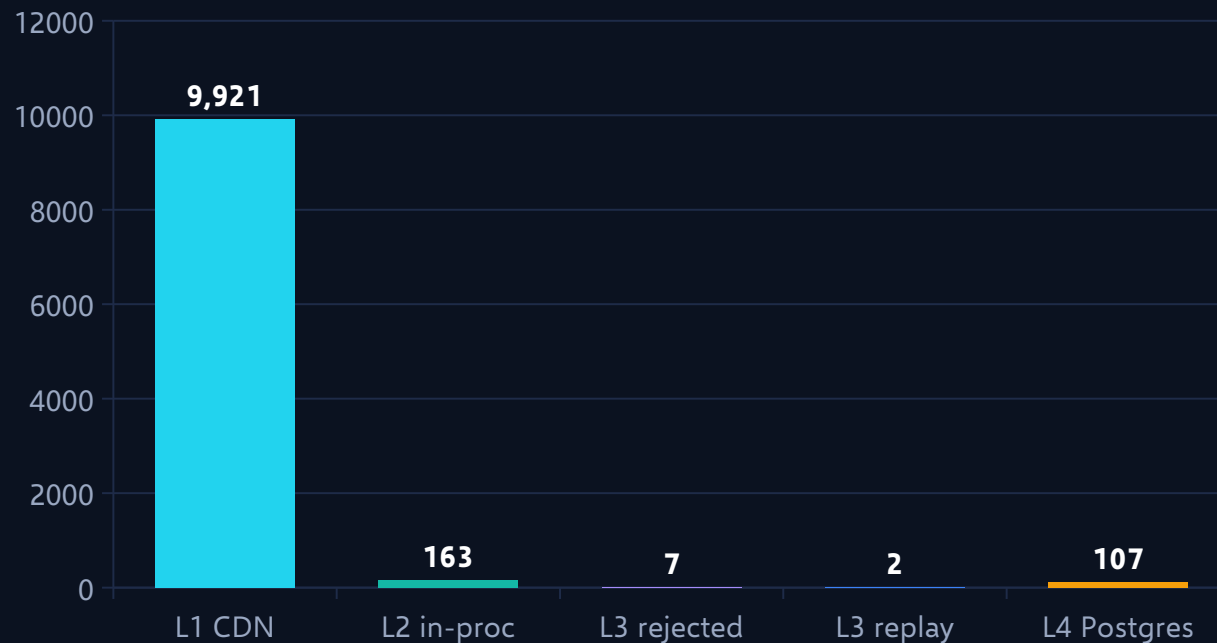
100

sold — exactly

415 / 0

invariant checks / violations

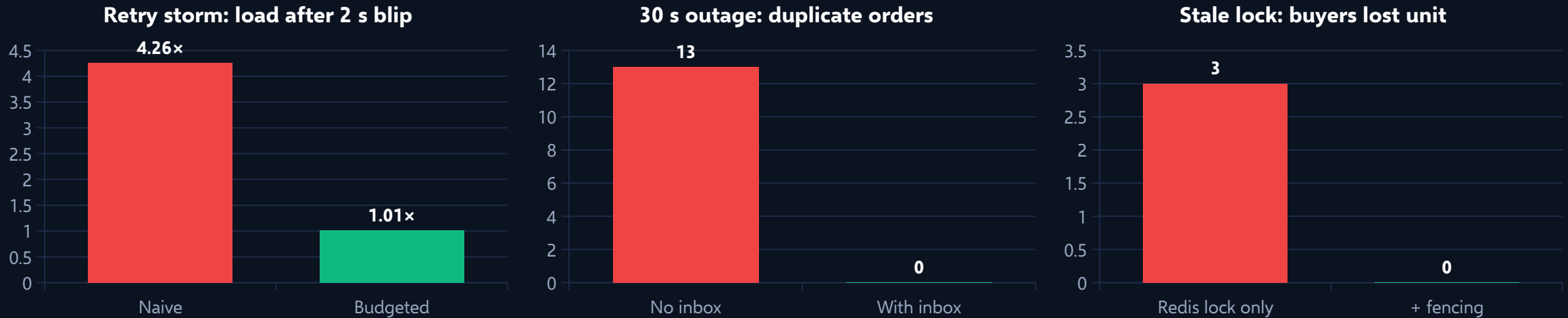
Where 10,200 requests were answered



Units sold (stock 100)



Proof: resilience experiments



97.2%

goodput, recovered < 1 s (naive never recovered)

2,000 → 1

cache misses → DB reads (singleflight)

57

max ops on one key (16 salted pools, 10k buyers)

1 → 1,902 ms

p99 closed-loop (lies) vs open-loop (truth)

30 s Order Service outage — recovery timeline



Architecture decision records (10)

ADR 001**Cache-Hit Ladder for latency and load****ADR 006****Authorize-then-capture payments****ADR 002****Inventory as unit rows + SKIP LOCKED****ADR 007****Transactional outbox + orchestrated saga****ADR 003****Redis token pool as admission control****ADR 008****Pessimistic for stock, optimistic elsewhere****ADR 004****PostgreSQL (SQL) over NoSQL for money & stock****ADR 009****Shedding, retry budget, fencing, inbox****ADR 005****Sync reserve/authorize, async order/ship/notify****ADR 010****Own cell + open-loop load testing**

Tools we used



Draw.io

HLD, LLD, ER diagrams



StarUML / UML

class, sequence, state



MySQL Workbench

ER from schema_mysql.sql



dbdiagram.io

salestorm.dbml ER



Swagger / OpenAPI

live API docs



Postman

collection + idempotency tests



Locust

open-loop load tests



JMeter

throughput test plan



GitHub

repo, PRs, history



GitHub Pages

live documentation site

Team ownership — everyone can answer anything

S1

System Architect

HLD, scalability, deployment,
cell

ADR 001 • 003 • 004 • 005 • 010

S2

LLD & Design Engineer

Classes, SOLID, patterns,
sequences, states

UML • 06_SOLID • 07_Patterns

S3

Data & API Engineer

22 tables, OpenAPI,
outbox/inbox, payments

ADR 004 • 006 • 007

S4

Reliability Engineer

Concurrency, 14 failure modes,
security, proof

ADR 002 • 008 • 009

LIVE DEMO • DOCS • CODE

Thank you

Caches can say **NO**. Only the database can say **YES**.

Live site

<https://sivaangayarkanni.github.io/SalesStrom-Ecommerce-platform/>

Swagger API

<https://sivaangayarkanni.github.io/SalesStrom-Ecommerce-platform/api.html>

Repository

github.com/sivaangayarkanni/SalesStrom-Ecommerce-platform

Questions?



Scan for the live site

100 sold • 0 oversold
0 double charges • 0 lost orders